AI Forward Deployed Engineer

The AI Architect & FDE Course



Week 2 Day 4

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer

AI ENGINEER

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe how an FDE builds AI solutions to solve business problems
- [x] Use evaluations and observability in Langfuse to drive business outcomes
- [x] Build agents that have memory with Agentic RAG over MCP
- [x] Create durable agents in LangGraph with a human in the loop

By the end of today, you'll be able to:

- [ ] Distinguish between agent teams, swarms and fleets
- [ ] Confidently run a swarm of agents with LangGraph
- [ ] Handle rate limits, connection caps and retries at scale
- [ ] Measure quality, cost and speed on an eval scoreboard

Recap

Agents can survive a crash and wait for a human



Durable execution

A run carries on where it stopped



Checkpoints and threads

State saved after every step



Never pay twice

The claim ID key makes retries safe



Human in the loop

interrupt() waits for a sign-off

Teams, swarms and fleets



Team

A few agents coordinated on one task

Claude Code agent teams
A lead and teammates share a task list



Swarm

Many agents working in parallel

Kimi Agent Swarm
Up to 100 sub-agents working at once



Fleet

Every agent a business runs and governs

LangSmith Fleet
One place to manage every agent

The most immediate goal is speed; the more interesting goal is complexity of task

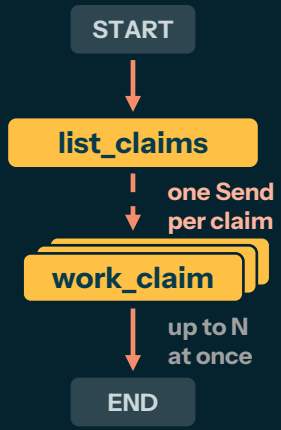
A LangGraph graph with no LLM

LangGraph doesn't need to have anything to do with agents

```python
class Dispatch(TypedDict):
    claims: list[str]
    results: Annotated[list, operator.add]

def fan_out(state: Dispatch):
    return [Send("work_claim", claim_id)
            for claim_id in state["claims"]]

builder = StateGraph(Dispatch)
builder.add_node("list_claims", list_claims)
builder.add_node("work_claim", work_claim)
builder.add_edge(START, "list_claims")
builder.add_conditional_edges(
    "list_claims", fan_out, ["work_claim"])
builder.add_edge("work_claim", END)
dispatcher = builder.compile()
dispatcher.invoke({}, {"max_concurrency": 5})
```



Like the payout tool, the dispatcher is business logic that belongs in code

Limits, caps and retries

What we run into with 25 claim agents at once



Rate limits
Too many calls at once get a 429

→ `RetryPolicy`
Waits with backoff and tries again



Database connections
Supabase's free pooler allows 15

→ `max_concurrency`
Caps how many agents run at once



Retries
A timed-out call may have paid

→ **The claim ID key**
The backend refuses to pay twice

Today's lab

In labs/week2day4.ipynb and its free twin



A dispatcher with no LLM

Fan out, cap and retry with a stand-in

A swarm of claim agents

The payout agent works the harder claims in parallel

Scoring the run

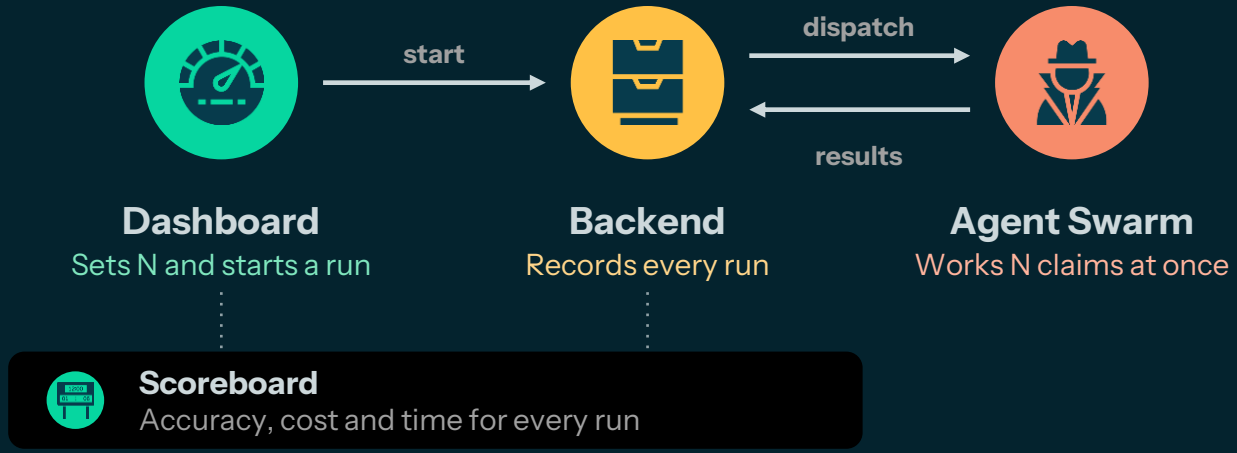
Quality, cost and time, grouped as one run in Langfuse

Five agents or 25

Time and cost at N=5 and N=25

Upgrading the product

Start a dispatch from the dashboard and compare the runs



Also in this build: an eval runner that records cost and time per claim

Building it with Claude Code

The change is written up in labs/week2day4_product_SWARM.md

1. **Copy the brief into docs**
   `cp labs/week2day4_product_SWARM.md docs/SWARM.md`
2. **Ask Claude Code for a plan**
   Please plan the swarm in docs/SWARM.md
3. **Let it build and commit each phase**
   Please implement the plan for the swarm
4. **Start a dispatch from the dashboard**
   Set N to 3, start it, then run an evaluation
5. **Or check out my version**
   `git checkout week2day4-start or git checkout week2day4-end`

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer AI ENGINEER

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course

